



HierRTLBench: Evaluating LLM-Based Verilog RTL Generation Under Context-Window Constraints

Yashwant Rajesh^{*1}, Karthikeya Patana^{*2}, Durga Saranyu^{*3}, Aathira Sunil^{*4}
 Sri Parameswaran^{#5}, Kamesh Chandrasekar^{†6}, Soumya Joshi^{*7}, Paresh Saxena^{*8}
^{*}Department of EEE, BITS Pilani-Hyderabad Campus, Hyderabad 500078, India
[#]Department of Electrical and Computer Engineering, University of Sydney, Australia
[†]AMD Inc., India

{¹f20230269, ²f20231115, ³2025420069, ⁴p20250480, ⁷soumyaj, ⁸psaxena}@hyderabad.bits-pilani.ac.in
⁵sri.parameswaran@sydney.edu.au, ⁶kamesh.chandrasekar@amd.com

Abstract

We present **HierRTLBench** (Hierarchical Register-Transfer Level Benchmark), a benchmark suite and evaluation framework for measuring how well large language models (LLMs) can automatically write hardware circuit description code (HDL- Hardware Description Language). While existing benchmarks largely focus on small, isolated circuit pieces, they have not been specifically designed to surface the failure modes that arise when token-budget constraints interact with realistic, hierarchically complex hardware designs. HierRTLBench is designed to begin characterizing these failure modes and to provide a diagnostic vocabulary that could inform future model training and benchmark design. Our contributions are: (1) a benchmark suite spanning processors, caches, cryptographic cores, DSP units, and communication interfaces; (2) a hierarchy-aware decomposition workflow that breaks large designs into context-safe submodule prompts (≤ 120 tokens each), significantly reducing output truncation during HDL generation and keeping per-query inference cost bounded because token efficiency is an active economic concern in production LLM deployment, not merely a structural one; (3) an automated multi-layer verification pipeline using the Icarus Verilog open-source simulator, demonstrating that reductions in truncation errors do not translate into corresponding improvements in end-to-end functional pass rates; and (4) a six-class RTL error taxonomy evaluated across VeriGen 2 billion, 6 billion, and 16 billion parameter models, where the 6B model collapses to a 0% pass rate, exhibiting behaviors consistent with possible non-monotonic scaling effects [15], including repeated generation of unrelated external macro symbols. Syntax pass rates across the submodule runs are 44% (2B), 0% (6B), and 54% (16B); end-to-end functional pass rates drop to 30%, 0%, and 28% respectively. Our infrastructure is released openly to support reproducible LLM-hardware research.

1 Introduction

The prevailing assumption in Large Language Model (LLM)-based hardware design is that increasing model size and extending context windows will eventually close the gap between model-generated

and human-written HDL. Larger models score better on simple benchmarks; wider context windows allow longer prompts. In this paper we set out to test whether that assumption holds on complex, hierarchical designs that actually stress it.

HDL development remains labor-intensive, expensive, and error-prone. A single processor design can require months of expert engineering. *Verilog* is the language used to describe digital circuits at the *register-transfer level* (RTL) which specifies how data moves between storage elements cycle by cycle. LLMs fine-tuned on Verilog corpora [1, 4, 6] have shown promise as automated RTL code writers. However, three structural limitations prevent their practical use.

(1) **Token-window bottleneck:** LLM’s context window can be thought of as its working memory; it can only “see” a fixed amount of text at once and it includes both the prompt and the generation. Token usage incurs direct monetary and energy costs during LLM inference, making context efficiency not just a correctness concern but a direct economic one; token-budget-aware prompting has been shown to reduce per-query expenses by up to 59% while preserving accuracy [18]. Open-source HDL models such as VeriGen [1] inherit CodeGen’s hard 2048-token limit [10]. A prompt describing a 5-stage pipelined RISC-V processor already consumes ≈ 550 tokens, leaving only ≈ 1498 for generation, but a complete processor needs ≈ 4000 – 6000 tokens of Verilog. The model runs out of space and stops mid-sentence in the code, silently producing broken output.

(2) **Shallow benchmarks:** Existing benchmarks such as VerilogEval [3] test only simple, isolated circuit components (e.g., a single multiplexer). They have not been designed to evaluate realistic hierarchical designs like multi-stage pipelines, cryptographic state machines, or cache controllers, which are central to modern hardware design workflows, leaving the failure modes that arise at these complexity levels largely uncharacterized.

(3) **No systematic error analysis:** Published evaluations report only pass/fail percentages without classifying *why* failures occur, making it impossible to diagnose model weaknesses or guide improvements. Moreover, when such error analysis does happen, it is typically performed internally by individual model developers using ad hoc, unpublished criteria, making results incomparable across models and studies. A shared, openly released benchmark and taxonomy allows failure modes to be diagnosed consistently across models rather than re-derived independently each time.

If the context window is the primary bottleneck, then decomposing large designs into submodule prompts small enough to fit inside



the window should close the functional gap in the majority of cases. We built HierRTL Bench to test exactly this hypothesis and to measure what remains broken when the context pressure is relieved. Every submodule prompt stays ≤ 120 tokens, significantly reducing truncation compared to monolithic end-to-end HDL generation. An automated pipeline compiles every generated file, categorizes every error under a six-class taxonomy, and runs functional tests where compilation succeeds.

Evaluating VeriGen-2B, 6B, and 16B over 150 total verification runs, we find: (i) the 6B model is catastrophically worse than 2B, with 0% functional pass rate; (ii) larger models do not automatically improve results; (iii) Token-budget pressure is likely a contributing factor in approximately 42.6% of observed outputs, based on truncation rates across all three models even after decomposition, though further controlled experiments are needed to confirm its causal role in functional failures; and (iv) complex modules (AES encryption, FFT signal processing, pipelined RISC-V CPU) resist all model sizes.

Section 7 discusses why semantic failures in FSM sequencing, cryptographic constants, and pipeline coordination persist after truncation is reduced, with implications for training-data curation and future RTL generation design

2 Related Work

LLM-based hardware code generation. VeriGen [1] trains five CodeGen model variants (345M–16B parameters) on 400 MB of Verilog code, evaluating on 17 hand-designed and 164 simple benchmark problems. VerilogEval [3] extends evaluation to 156 problems using automated simulation. RTLcoder [4] shows a 7B model can match GPT-3.5-turbo on hardware tasks through curated training. GPT4AIGChip [9] explores LLM-driven AI accelerator design automation. All share a common limitation: their benchmarks consist of single, simple modules with small expected outputs, missing the hierarchical complexity of real chip design.

Hierarchical and agent-based generation: ROME [5] introduces multi-step hierarchical prompting for chip design. HiVeGen [2] extends this with code retrieval and design-space exploration, reporting 45.24% runtime savings over direct GPT generation but targets large closed-source models (GPT-4/O1) rather than the constrained open-source models we study. Neither work provides a general benchmark across heterogeneous design categories or a systematic error classification.

Hardware-specific benchmarks: RTLLM [7] provides 30 benchmark designs; ChipChat [8] explores conversational RTL design at the CPU level. Our work is orthogonal: we provide evaluation *methodology* and *infrastructure* applicable to autoregressive hardware models such as GPT-based, Claude-based, and open-source code generation systems, rather than proposing a new generative technique.

Post-generation repair: RTLfixer [14] addresses the complementary problem of automatically correcting syntax errors in LLM-generated Verilog using Retrieval-Augmented Generation (RAG) and ReAct prompting, correcting 98.5% of compile-time errors on the VerilogEval-syntax dataset. Their analysis finds that approximately 55% of errors in LLM-generated Verilog are syntax-related

which is a figure consistent with the high F1 failure rates we observe. Our work is orthogonal: while RTLfixer repairs errors post-generation on simple isolated modules, HierRTL Bench classifies error types across hierarchically complex designs before any repair stage, exposing deeper failure modes like semantic mismatch, FSM sequencing errors, and cryptographic constant hallucination that survive syntax correction and that no repair framework has addressed at realistic hardware complexity.

3 HierRTL Bench Design

3.1 Module Selection

HierRTL Bench spans 14 RTL modules across 7 hardware design categories (Table 1), covering the full range from simple combinational logic (gates and adders) to complex pipelined processors and cryptographic engines. Modules were selected to cover the principal hardware abstraction layers from combinational primitives to multi-stage pipelines and algorithmic accelerators, mirroring the complexity diversity of benchmark suites such as SPEC CPU2017 and MiBench when translated to equivalent hardware workloads.

Table 1: HierRTL Bench Module Summary (50 submodules)

ID	Module	Category	No. of Submodules
M01	Param. N-bit ALU	Datapath	4
M02	32-Entry Register File	Memory	1
M03	UART TX (8-N-1)	Communication	3
M04	Multi-Cycle RISC CPU	CPU	6
M05	Hardwired Control Unit	Control	2
M06	5-Stage Pipelined RISC-V	CPU	6
M07	Direct-Mapped Cache	Memory	3
M08	Round-Robin Arbiter	Interconnect	2
M09	AES-128 Encryption Core	Cryptography	9
M10	SHA-256 Hash Core	Cryptography	4
M11	CRC-32 Engine (IEEE 802.3)	Error Detection	2
M12	8-Point Fixed-Point FFT	DSP	4
M13	16×16 Matrix Multiplier	DSP/ML	2
M14	Hardware Bubble Sort	Algorithm	2
Total		7 categories	50

3.2 Submodule Decomposition

Each module is decomposed into independently generated RTL submodules following standard hardware design practice: leaf combinational units (pure logic with no clock), registered FSMs, memory arrays, arithmetic units, and pipeline registers. Top-level integration wrappers that instantiate and connect submodules are excluded from automated generation, since VeriGen operates primarily as a code-completion model and showed limited reliability for hierarchical module integration tasks. These top-level integration modules were therefore implemented independently. Each generated submodule receives a prompt bounded to ≤ 120 tokens (mean: 73 tokens), giving the model ≥ 1900 tokens for generation which is enough for any individual component.

Every submodule additionally receives a dedicated testbench, while higher-level integration testbenches verify correct interaction among connected submodules.

4 Hierarchical Decomposition Workflow

4.1 Why Monolithic Generation Fails

To illustrate the problem our decomposition solves, consider RISC-V Pipeline (M06) : its monolithic prompt consumes ≈ 550 tokens, leaving only 1498 for generation. But a correct 5-stage pipelined processor requires 4000–6000 tokens of Verilog, so the model can

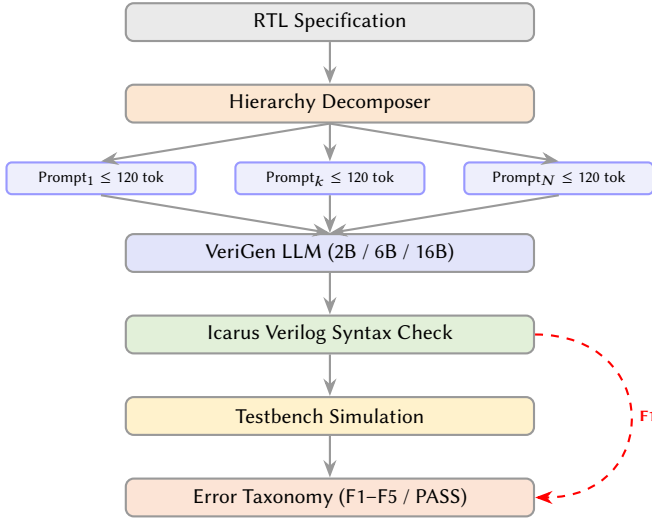


Figure 1: HierRTL Bench evaluation workflow

at best write one-quarter of the needed logic before running out of space, stopping abruptly mid-expression or in the middle of an always block (the hardware equivalent of interrupting a sentence half-way through). Table 2 quantifies this pressure across representative modules.

Table 2: Token Budget Analysis for Representative Modules

Module	Prompt	Output Budget	Req.	Risk
N-bit ALU	80	1968	~300	Safe
UART TX	200	1848	~800	Safe
CRC-32 Engine	280	1768	~1200	Borderline
SHA-256 Core	480	1568	~3500	Critical
AES-128 Core	420	1628	~4200	Critical
Multi-Cycle CPU	450	1598	~5000	Critical
5-Stage RISC-V	550	1498	~6000	Critical

After decomp.: all submodules ≤ 120 tok $\Rightarrow \geq 1900$ output budget.

After hierarchical decomposition, 72 out of 150 generated files terminated cleanly with a complete endmodule statement corresponding to the intended module definition. For the 6B model, several generations additionally appended unrelated continuations (most commonly autogenerated testbenches) after successfully completing the target RTL module. These cases were conservatively excluded from the strict completion metric, although 14 such files still contained a syntactically complete primary module before continuation drift occurred. Hierarchical decomposition substantially reduced truncation-related failures, isolating most remaining errors to semantic mistakes in the generated logic. An additional advantage of hierarchical decomposition is evaluation parallelism. Since submodules are generated and verified independently, compilation and simulation jobs can execute concurrently across compute nodes. In contrast, monolithic RTL generation requires sequential end-to-end compilation and debugging, increasing overall verification latency for complex designs. Modules marked *Critical* produce incomplete logic under monolithic generation, while *Borderline* modules risk partial always-block truncation. Although hierarchical decomposition behaved as expected at the structural level, sharply reducing truncation-related failures, end-to-end functional pass rates remained unexpectedly low at 30% (2B), 0% (6B), and 28%

(16B). This disconnect between structural improvement and functional stagnation forms the central empirical motivation for the error taxonomy presented in Section 6, suggesting that context-window limitations are not the primary barrier to reliable RTL generation. More fundamental semantic and architectural failures persist even after truncation is mitigated.

5 Experimental Setup

Models: We evaluate three VeriGen checkpoints (2B, 6B, and 16B parameters) obtained from the official repository accompanying the VeriGen paper [1]. VeriGen fine-tunes CodeGen-family autoregressive code-generation models on Verilog corpora, making these checkpoints broadly comparable to other open-source code-generation LLMs evaluated in recent RTL-generation studies. The 2B, 6B, and 16B variants correspond to increasing parameter scales within the same underlying CodeGen-derived architecture. All models use the CodeGen architecture [10] with a 2048-token context window. Inference is performed using greedy decoding (temperature = 0, $n = 1$) to ensure deterministic generation.

Infrastructure: All generation jobs were executed on a High Performance Computing (HPC) cluster equipped with NVIDIA H100 80GB SXM5 GPUs. Workloads were managed using the SLURM (Simple Linux Utility for Resource Management) scheduler, which handles distributed job allocation, GPU scheduling, and resource orchestration across compute nodes. The software stack used Python 3.10 with PyTorch (CUDA 12.1) and HuggingFace Transformers/Accelerate.

Verification: Syntax checking uses Icarus Verilog (iverilog) v11 [11]. A Python runner iterates all .v files, invokes iverilog with full error capture, and classifies output against the taxonomy in Section 6. Functional testbenches run with vvp; the runtime simulation engine of Icarus Verilog that executes the compiled HDL and verifies circuit behavior against expected outputs; simulations exceeding 10k cycles are classified as TIMEOUT.

Prompts: Each of the 50 submodule prompts uses the format established in [1]: a natural-language comment describing function and interface, a complete module header with port declarations, and a partial RTL skeleton.

6 Results and Error Taxonomy

6.1 Error Taxonomy

Rather than simply grouping errors after evaluation, the taxonomy is designed to identify the dominant failure modes that persist even after context-window truncation is reduced. Together they answer the question left open by the structural improvement: *what is actually wrong with the generated code?*

We define six mutually exclusive outcome classes ordered by verification stage (Table 3). Think of these as increasingly deep levels of correctness: F1 fails at the compiler; F5 compiles and runs but produces wrong answers.

Root causes of F1 failures. Analysis of F1 errors across all 150 runs reveals five iverilog-detectable subtypes:

- *Duplicate declarations* (dominant in 2B and 6B): The model enters a degenerate repetition loop, appending `_next` suffixes to signal names indefinitely, then re-declaring the original. The following output from `uart_tx_shift` (M03, 2B) shows many such cascading declarations:

Table 3: RTL Generation Error Taxonomy

Code	Stage	Description
PASS	Simulation	All testbench assertions pass
F1	Compile	Iverilog rejects at compile time
F2	Compile	Missing submodule type (undefined component)
F3	Simulation	Structurally incomplete (missing logic blocks)
F4	Simulation	Non-synthesizable construct (valid simulation, unmanufacturable)
F5	Simulation	Functional mismatch (wrong logic, bad FSM, incorrect operation)

```

reg [7:0] data_shift_reg; // line 13: declared
reg [7:0] data_shift_next_reg; // line 14
reg [7:0] data_shift_reg_next; // line 15
...
// continues for many more lines
reg [7:0] data_shift_reg_next_reg_next_reg_next_reg_next_reg_next; // from
    ↪ line 22 till the EOF : ERROR - redeclared

```

This pattern generated many compile errors in a single file; iverilog ultimately prints “I give up.”

- **Wire l-value violations:** A wire (a one-way connection, like a wire in a physical circuit) is incorrectly assigned inside a clocked always block (which should only drive reg variables). Observed in `cpu_regfile` (M04, 2B) and `rv_if_stage` (M06, 16B).
- **Undefined macros** (exclusive to 6B): The 6B model hallucinates entire framework-level constants like ‘`UartParityType`’, ‘`OR1200_OPERAND_WIDTH`’, and ‘`DMEM_MSB`’ which are symbols from unrelated open-source projects in its training data. This pattern is visible in many of the files from 6B variant.

Root causes of F2 failures. *Unknown module types:* The generated RTL is syntactically valid but instantiates submodules absent from the provided design hierarchy. VeriGen-2B produces three such cases: undefined `half_adder/full_adder` primitives in `alu_addsub` (M01), a missing `mux2` helper in `cpu_alu` (M04), and an undefined `sbox` module in `aes_shiftrows` (M09). VeriGen-16B contributes one case, where `fft_stage23` (M12) delegates computation to a hallucinated `fft_bits` module. VeriGen-6B produces no F2 failures because most generations fail earlier during syntax checking (F1).

6.2 Overall Pass Rates

Table 4 reports compilation and simulation success rates. Compilation success rate is the fraction of generated submodules that compile successfully with Icarus Verilog. Simulation success rate is the fraction of all generated submodules that pass functional testbenches end-to-end, indicating correct RTL behavior.

Table 4: Model Comparison: Error Counts and Pass Rates

Model	Error Count				Syntax	Simulation
	F1	F2	F3+F4	F5	Pass%	Pass%
VeriGen-2B	25	3	3	4	44	30
VeriGen-6B	50	0	0	0	0	0
VeriGen-16B	22	1	4	9	54	28
All	97	4	7	13	32.7	19.3

Note: $\text{Syntax Pass\%} = (50 - F1 - F2)/50$; $\text{Simulation Pass\%} = (50 - F1 - F2 - F3 - F4 - F5)/50$.

Indications of a Non-Monotonic Effect (6B Model): VeriGen-6B achieves 0% syntax and functional pass rates, performing substantially worse than both VeriGen-2B and VeriGen-16B. This behavior is qualitatively consistent with, though not conclusive evidence for, the U-shaped scaling effects reported in [15, 16]; as noted below, a difference in model architecture makes this reading one of

at least two equally plausible explanations. The 6B model frequently generated undefined external macros and unrelated testbench continuations, including symbols such as ‘`OR1200\_OPERAND\_WIDTH`’. These behaviors may reflect contamination or memorization effects involving macro-heavy Verilog repositories. Ignoring appended testbench continuations, 14/50 generations still contained a complete primary module before corruption or truncation occurred. The original VeriGen paper reports that the 6B backbone uses fewer attention heads than the 2B and 16B variants [1], so a weak-model or architectural explanation is at least as plausible as an inverse-scaling effect; our data cannot distinguish between these two possibilities, and we present the collapse descriptively rather than as evidence for either account.

16B vs. 2B: The 16B model achieves much better syntax pass rate (54% vs. 44%) but slightly lower functional pass rate (28% vs. 30%). Larger capacity reduces F1 compile errors but nearly doubles F5 functional failures (9 vs. 4): the model generates *structurally* correct code that produces *wrong answers*; particularly in FSM designs and cryptographic modules.

6.3 Per-Module Results

Table 5 shows pass rates per module (unit tests only).

Table 5: Per-Module Functional Pass Rate (successful generations / total generations for each model)

ID	Module	2B	6B	16B
M01	ALU	2/4	0/4	2/4
M02	Register File	0/1	0/1	0/1
M03	UART TX	0/3	0/3	0/3
M04	Multi-Cycle CPU	1/6	0/6	1/6
M05	Control Unit	1/2	0/2	2/2
M06	RISC-V Pipeline	2/6	0/6	2/6
M07	Cache	1/3	0/3	0/3
M08	Arbiter	1/2	0/2	1/2
M09	AES-128	2/9	0/9	2/9
M10	SHA-256	2/4	0/4	1/4
M11	CRC-32	1/2	0/2	1/2
M12	8-Pt FFT	1/4	0/4	0/4
M13	Matrix Multiply	0/2	0/2	1/2
M14	Bubble Sort	1/2	0/2	1/2

Zero-pass modules: Register File (M02) and UART TX (M03) fail across all three models in every evaluated generation. UART failures arise from incorrect FSM state-transition sequencing: models generate structurally plausible state machines but produce invalid `START→DATA→STOP` ordering. Register file failures stem from confusing when reads should be clocked versus unclocked: models write `rdata` inside clocked `always` blocks, introducing unintended cycle latency.

Control Unit (M05): VeriGen-16B achieves a perfect 2/2 score on M05. These modules primarily contain decoder-style control logic and large case structures commonly observed in Verilog training corpora.

AES-128 (M09) Widest error diversity: AES-128 spans submodules involving Galois-field arithmetic, S-box lookup structures, and key-scheduling logic, exercising multiple RTL reasoning patterns simultaneously. The models occasionally generate simpler XOR-style arithmetic correctly, but consistently fail on lookup-heavy affine transformation components such as `aes_sbox_lo`.

6.4 Code Coverage of Passing Modules

For every module that compiled and simulated successfully, coverage analysis was performed using the Synopsys VCS simulation

environment together with Verdi waveform and coverage analysis tools. VCS was used to execute RTL simulations and collect line, condition, toggle, and branch coverage metrics, while Verdi was used for detailed post-simulation coverage inspection and debugging. Table 6 summarizes coverage for the strongest-performing submodule per category. Passing submodules achieve consistently

Table 6: Representative Code Coverage for Passing Submodules

Module	Model	Line	Cond	Toggle	Branch
compare_swap	2B	96.9	100.0	99.2	83.3
compare_swap	16B	98.7	100.0	99.2	83.3
rv_hazard	2B	91.2	100.0	85.5	54.6
rv_hazard	16B	99.1	100.0	98.7	83.3
alu_flags	2B	98.9	87.5	98.1	50.0
alu_logic	16B	97.0	100.0	86.3	83.3
sha256_k_rom	2B	91.3	50.0	99.4	88.0
sha256_k_rom	16B	95.4	—	99.4	88.9
crc32_ctrl	2B	92.9	—	97.7	57.7
aes_gf_mul	2B	96.7	—	99.0	50.0
cache_ctrl	16B	99.6	50.0	76.9	95.2
fft_twiddle	2B	88.3	—	87.9	56.0

high coverage across multiple metrics. The observed coverage results increase confidence that successful simulations correspond to genuinely functional implementations rather than trivial or weakly tested outputs. Lower branch and condition coverage in some FSM-heavy modules suggests that while common execution paths are well validated, certain edge-case transitions and rare control paths remain underexplored. The “—” entries in conditional coverage arise from two distinct causes. In modules such as `crc32_ctrl` and `aes_gf_mul`, the generated RTL primarily consists of straight-line combinational datapath logic without condition-evaluable expressions, so VCS does not report meaningful conditional coverage metrics. In contrast, modules such as `sha256_k_rom` (16B) and `fft_twiddle` contained generated case-item width mismatches and improperly sized literals (e.g., unsigned decimal constants used instead of width-matched indexed values), which prevented reliable conditional coverage extraction despite successful simulation. These cases highlight that syntactically valid RTL may still contain structural inconsistencies that interfere with reliable coverage measurement.

6.5 Error Distribution and Token-Failure Correlation

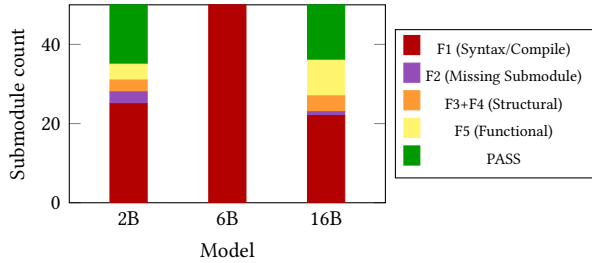


Figure 2: Error distribution per model across 50 leaf submodule evaluation runs per model (top-level and integration wrappers excluded)

Figure 2 reveals a clear qualitative shift: VeriGen-16B reduces the combined F1+F2 failure rate by 17.9% relative to 2B but increases F5 functional failures by 2.25×. This suggests that larger models

improve structural RTL generation without corresponding gains in semantic correctness. Modules requiring >300 tokens in monolithic form (AES, RISC-V, SHA-256, Multi-Cycle CPU) account for 54.5% of all F1 failures even after decomposition, indicating that hierarchical decomposition reduces truncation but does not fully resolve reasoning difficulties in semantically complex hardware designs.

7 Discussion

Section 6 posed the question: if decomposition removes truncation, why do functional pass rates remain low? The findings below gather the evidence and close the argument.

Decomposition is necessary but not sufficient. Breaking large designs into bounded submodule prompts reduces truncation and enables structural evaluation. However, it does not solve semantic generation accuracy. Clear, well-specified leaf-level submodules (e.g., `alu_shift`, `crc32_ctrl`) pass reliably in 2B and 16B, while submodules requiring memorized constant tables (AES S-box) or multi-cycle state management (UART FSM) fail consistently across all three models. Notably, `alu_shift` and `crc32_ctrl` both achieve PASS in 2B and 16B but fail at F1 in 6B, confirming that the 6B collapse is model-specific and not a reflection of submodule difficulty. The observed behavioral differences also suggest possible task specialization across model scales. VeriGen-2B appears comparatively more stable for localized RTL completion tasks and simpler datapath-style modules, while VeriGen-16B improves structural correctness but continues to exhibit semantic coordination failures in more complex hierarchical systems. However, the present benchmark size is not sufficient to support definitive deployment recommendations, and future work should evaluate interactive copilot-style RTL generation separately from autonomous full-project synthesis.

Possible training-data contamination effects in 6B. The 6B model frequently hallucinated macro symbols from unrelated external projects such as OpenRISC (`*OR1200_OPERAND_WIDTH`), which suggests that intermediate model sizes may be especially susceptible to training-data contamination [17]. The 2B model did not exhibit this specific macro-hallucination pattern in our evaluation runs. Beyond macro hallucination, a secondary failure mode was observed exclusively in 6B: unsolicited testbench generation. In 14 of the 50 leaf submodule files, the model completed the requested module correctly and then continued generating unrequested `_tb` modules within the same file; sometimes repeating the same testbench two or three times until the token limit was reached mid-generation and the output was cut off. This *intra-file module pollution* caused duplicate module declarations and fused `endmodulemodule` tokens that iverilog rejects at compile time, directly contributing to 6B’s F1 count. This finding motivates further investigation into how mid-size models develop hallucination behaviors and which training patterns contribute to these failures.

Integration failure amplification. Integration tests exhibit systematically worse pass rates than unit tests: of 42 integration-level runs (14 modules × 3 models), none of them produce a passing result. Any submodule F1 failure propagates to integration failure, creating a compounding effect. Future work should explore hybrid integration strategies in which failed generated submodules

are replaced with reference RTL implementations, enabling finer-grained measurement of how individual submodule errors affect the correctness of the assembled system.

Implications for model training. Our error taxonomy identifies three actionable improvement directions: (1) fine-tuning data should include more diverse, standalone Verilog files rather than macro-heavy framework repositories, so models learn to generate self-contained modules without external symbol dependencies; (2) training-corpus curation should explicitly filter out files containing multi-module definitions, particularly implementation-plus-testbench pairs in a single .v file to prevent 6B-style intra-file pollution and unsolicited testbench generation; and (3) prompt engineering with explicit port-type annotations may reduce wire l-value violations.

Limitations of Completion-Only RTL Models. During integration-level evaluation, we observed that VeriGen frequently ignored explicit natural-language instantiation directives inside top-level prompts. For example, prompts specifying:

```
Instantiate alu_addsub #(N)(...),
            alu_logic   #(N)(...),
            alu_shift   #(N)(...),
            alu_flags   #(N)(...).
```

did not result in corresponding module instantiations in the generated RTL. Instead, the model attempted to regenerate local arithmetic and control logic directly inside the top-level module body. This behavior reflects a limitation of VeriGen’s completion-only training objective. Unlike instruction-tuned conversational LLMs such as ChatGPT [12] or Claude [13], VeriGen is trained primarily for next-token RTL completion rather than hierarchical planning. Consequently, VeriGen can successfully continue local RTL patterns inside incomplete modules, but struggles to perform global structural reasoning required for top-level system integration. In practice, several top-level integration wrappers were constructed manually to maintain benchmark consistency, while all submodule implementations were generated by the evaluated models. Future RTL generation systems may require instruction tuning or graph-level architectural reasoning to reliably compose large multi-module hardware systems. Taken together, the evidence presented in this paper suggests that within the completion-only, CodeGen-derived model family we evaluate, neither model scale nor context-window length appears to be the primary limitation in LLM-based RTL generation at realistic hardware complexity levels. Instead, dominant constraints within this model family arise from training-data quality and limited domain-specific semantic reasoning involving FSM sequencing, cryptographic constant handling, and pipeline coordination. We do not claim this finding generalizes to instruction-tuned models, which may reason about hierarchical structure differently.

Scope and Limitations. Our findings rest on a single, dated model family (VeriGen 2B/6B/16B, CodeGen-based, completion-only, 2048-token window); modern instruction-tuned models may behave differently and were outside the scope of this study. All results reflect a single greedy-decoding run per submodule ($n = 1$); we did not measure variance across repeated samples. Extending HierRTLBench to instruction-tuned models (e.g., GPT-4, Claude) with repeated sampling is a natural next step and our primary direction for future work.

Our taxonomy also does not currently capture lint-level design quality issues such as unintended latch inference or multiply-driven nets, which we leave for future extension of the error classification.

8 Conclusion

We presented **HierRTLBench**, a hierarchical hardware benchmark suite and evaluation framework for LLM-based Verilog generation. The benchmark covers 14 diverse modules of different complexity decomposed into 50 submodules spanning processors, caches, cryptographic cores, DSP accelerators, and communication interfaces. Evaluating VeriGen-2B, 6B, and 16B over 150 leaf submodule verification runs, we found:

- (1) The 2048-token context limit makes monolithic generation infeasible for 4 out of the 14 modules; hierarchical decomposition drastically reduces structural truncation from 100% under monolithic generation down to 52% across decomposed outputs (strict metric: file terminates cleanly with no extraneous content). If we additionally credit the 14 VeriGen-6B submodules that completed the target module correctly before appending an unsolicited testbench, truncation falls further to 42.6% (lenient metric). We report both to make explicit how the testbench-pollution cases are counted; all other results in this paper use the strict metric unless noted.
- (2) VeriGen-6B exhibits a complete collapse in functional correctness (0% pass rate), driven not by token-pressure limitations alone, but by what appears to be training-data contamination effects [17], such as undefined macro generation and unsolicited intra-file testbench insertion. This behavior aligns with the non-monotonic scaling effect reported in [15], where intermediate-scale models can amplify training-data pathologies instead of mitigating them.
- (3) VeriGen-16B achieves a higher syntax pass rate (54% vs. 44%) but produces roughly twice as many F5 functional failures (9 vs. 4), suggesting larger capacity may improve structural correctness while being associated with more semantic failures which is a qualitative shift rather than an overall improvement.
- (4) Complex modules involving FSM sequencing (UART, CPU control), cryptographic constant tables (AES, SHA-256), and multi-stage pipeline coordination (RISC-V) resist all three model sizes.
- (5) Passing submodules achieve strong line coverage, but branch and condition coverage remain low, revealing testbench gaps in corner-case and edge-state exercise that leave meaningful portions of the generated logic unvalidated.

The HierRTLBench error taxonomy (F1–F5) provides structured vocabulary for diagnosing RTL generation systems, enabling targeted improvements in fine-tuning data curation, prompt engineering, and post-generation repair techniques. Furthermore, the identification of intra-file module pollution as a distinct failure class unique to intermediate-scale models opens a new direction for training-corpus filtering and generation-boundary enforcement in future LLM-based hardware synthesis pipelines.

Acknowledgments

This work is partially supported by the research project sponsored by Xilinx-AMD. Opinions and conclusions presented are solely those of the authors and are not representative of AMD’s roadmap.

References

- [1] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "VeriGen: A large language model for Verilog code generation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 3, Apr. 2024.
- [2] J. Tang, J. Qin, K. Thorat, C. Zhu-Tian, Y. Cao, Y. Zhao, and C. Ding, "HiVeGen: Hierarchical LLM-based Verilog generation for scalable chip design," in *Proc. IEEE ICLAD*, 2025, pp. 30–36.
- [3] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. IEEE/ACM ICCAD*, 2023.
- [4] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "RTLCoder: Outperforming GPT-3.5 in design RTL generation," in *Proc. IEEE LLM Aided Design Workshop (LAD)*, 2024.
- [5] A. Nakkab, S. Q. Zhang, R. Karri, and S. Garg, "Rome was not built in a single step: Hierarchical prompting for LLM-based chip design," in *Proc. ACM/IEEE MLCAD*, 2024.
- [6] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "ChipGPT: How far are we from natural language hardware design," arXiv:2305.14019, 2023.
- [7] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An open-source benchmark for design RTL generation with large language models," arXiv:2308.05345, 2023.
- [8] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-Chat: Challenges and opportunities in conversational hardware design," in *Proc. ACM/IEEE MLCAD*, 2023.
- [9] Y. Fu, Y. Zhang, Z. Yu, S. Li, Z. Ye, C. Li, C. Wan, and Y. Lin, "GPT4AIGChip: Towards next-generation AI accelerator design automation via LLMs," in *Proc. IEEE/ACM ICCAD*, 2023.
- [10] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis," arXiv:2203.13474, 2022.
- [11] S. Williams, "The Icarus Verilog compilation system," <https://github.com/steveicarus/iverilog>, 2023.
- [12] OpenAI, "ChatGPT: Optimizing language models for dialogue," <https://openai.com/blog/chatgpt>, 2023.
- [13] Anthropic, "Claude: An AI assistant by Anthropic," <https://www.anthropic.com/claude>, 2024.
- [14] Y. Tsai, M. Liu, and H. Ren, "RTLFixer: Automatically fixing RTL syntax errors with large language models," in *Proc. 61st ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [15] I. R. McKenzie et al., "Inverse scaling: When bigger isn't better," *Trans. Mach. Learn. Res.*, 2023, arXiv:2306.09479.
- [16] J. Wei, Y. Tay, and Q. V. Le, "Inverse scaling can become U-shaped," arXiv:2211.02011, 2023.
- [17] M. Riddell, A. Ni, and A. Cohan, "Quantifying contamination in evaluating code generation capabilities of language models," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, vol. 1 (Long Papers), pp. 14116–14137, Aug. 2024.
- [18] T. Han, Z. Wang, C. Fang, S. Zhao, S. Ma, and Z. Chen, "Token-budget-aware LLM reasoning," in *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 24842–24855, Vienna, Austria, Association for Computational Linguistics, 2025, arXiv:2412.18547.

A Artifact Appendix

This artifact contains the complete benchmark suite, generation scripts, pre-generated Verilog outputs, and testbenches for replicating the experimental results of HierRTLBench. It compares unified and hierarchical RTL generation strategies across 14 hardware modules using three open-weight VeriGen checkpoints (2B/6B/16B). Reviewers can validate the pre-generated outputs with Icarus Verilog syntax checks (no GPU required) or regenerate them from scratch on an NVIDIA GPU.

A.1 Check-List (meta-information)

- **Program:** HierRTLBench
- **Compilation:** Python 3.9+ with PyTorch 2.1+ (CUDA 12.x)
- **Binary:** Python scripts and shell scripts (no compiled binaries)
- **Model:** VeriGen 2B/6B/16B, open-weight, Apache-2.0 licensed
- **Dataset:** None; 14 module specifications are authored prompts, not a downloaded dataset
- **Run-time environment:** Ubuntu 20.04/22.04, CUDA 12.x (GPU needed only for regeneration)
- **Hardware:** CPU only for evaluation; NVIDIA GPU with ≥ 16 –80 GB VRAM for regeneration
- **Output:** Generated .v files and a syntax/functional pass-rate summary, per Table 4
- **Experiments:** See Section A.6
- **Disk space required (approximately):** 6 MB (repo and cached outputs); 50 GB (model weights, on demand)
- **Time needed to complete experiments (approximately):** under 5 minutes cached; 6–8 GPU-hours for full regeneration
- **Publicly available?:** Yes (via GitHub)

A.2 How to access

Our source code is available on GitHub:
<https://github.com/durgasanyu/HierRTLBench>

A.3 Hardware dependencies

- A modern CPU is sufficient for the cached-output evaluation path
- An NVIDIA GPU is required only for regeneration: ≥ 16 GB VRAM (2B), ≥ 24 GB (6B), ≥ 80 GB (16B); an H100 or A100 is recommended
- Python 3.9+
- PyTorch 2.1+

A.4 Software dependencies

- Linux-based operating system (Ubuntu recommended)
- transformers 4.38+, accelerate 0.27+, sentencepiece 0.1.99+
- Icarus Verilog (iverilog) 11.0+
- A Dockerfile is provided for a self-contained, CPU-only evaluation environment; GPU regeneration instead uses `setup_env.sh`

A.5 Description

VeriGen checkpoints are open-weight and hosted on Hugging Face, downloaded automatically on first use. Inference

settings are fixed across all experiments: `max_length=2048`, greedy decoding (`temperature=0`, `do_sample=False`), one sample per submodule. Generated outputs are pre-included under `vgen_project/verigen_out/{2B,6B,16B}/` (hierarchical) and `vgen_project/unified_outputs/{2B,6B,16B}/` (unified), together with testbenches under `vgen_project/verigen_testbenches/`.

A.6 Experiment workflow

After cloning the repository and installing dependencies, the complete cached-output evaluation can be run with a single command:

```
docker build -t hierrtlbench-ae .
docker run --rm hierrtlbench-ae bash run_evaluation.sh --
    ↪ cached
```

or, without Docker:

```
pip install -r requirements.txt
bash run_evaluation.sh --cached
```

Progress and the syntax pass-rate summary are printed directly to the terminal. Full regeneration, which requires a GPU, is triggered with `run_evaluation.sh -full-regen` and reruns `verigen_runner.py/run_unified_verigen.py` for each of the three checkpoints before re-evaluating with the same script.

A.7 Evaluation and expected results

As a smoke test, the following should print PASS:

```
docker run --rm hierrtlbench-ae \
    iverilog -tnull vgen_project/verigen_out/2B/m02_regfile/
    ↪ regfile_mem.v \
    && echo PASS
```

Running the full cached evaluation reproduces Table 4: syntax pass rates of 44% (2B), 0% (6B), and 54% (16B), and functional pass rates of 30%, 0%, and 28% respectively. Pass rates may vary by ± 1 submodule ($\pm 2\%$) across runs due to minor library/driver version drift (e.g. iverilog version, PyTorch/CUDA build) — generation itself is deterministic. All outputs are written under `vgen_project/verigen_out/` and `vgen_project/unified_outputs/`.

A.8 Experiment customization

Key parameters are configured directly in the runner scripts: model size (`-model 2B/6B/16B`), output directory (`-output_dir`), a single submodule for a fast sanity check (`-idx N`, enumerated via `-list`), and sampling temperature (`GEN_CFG["temperature"]` in `verigen_runner.py`).

A.9 Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <http://cTuning.org/ae/submission-20201122.html>
- <https://github.com/ml-eda/artifact-evaluation/>